Containerisation

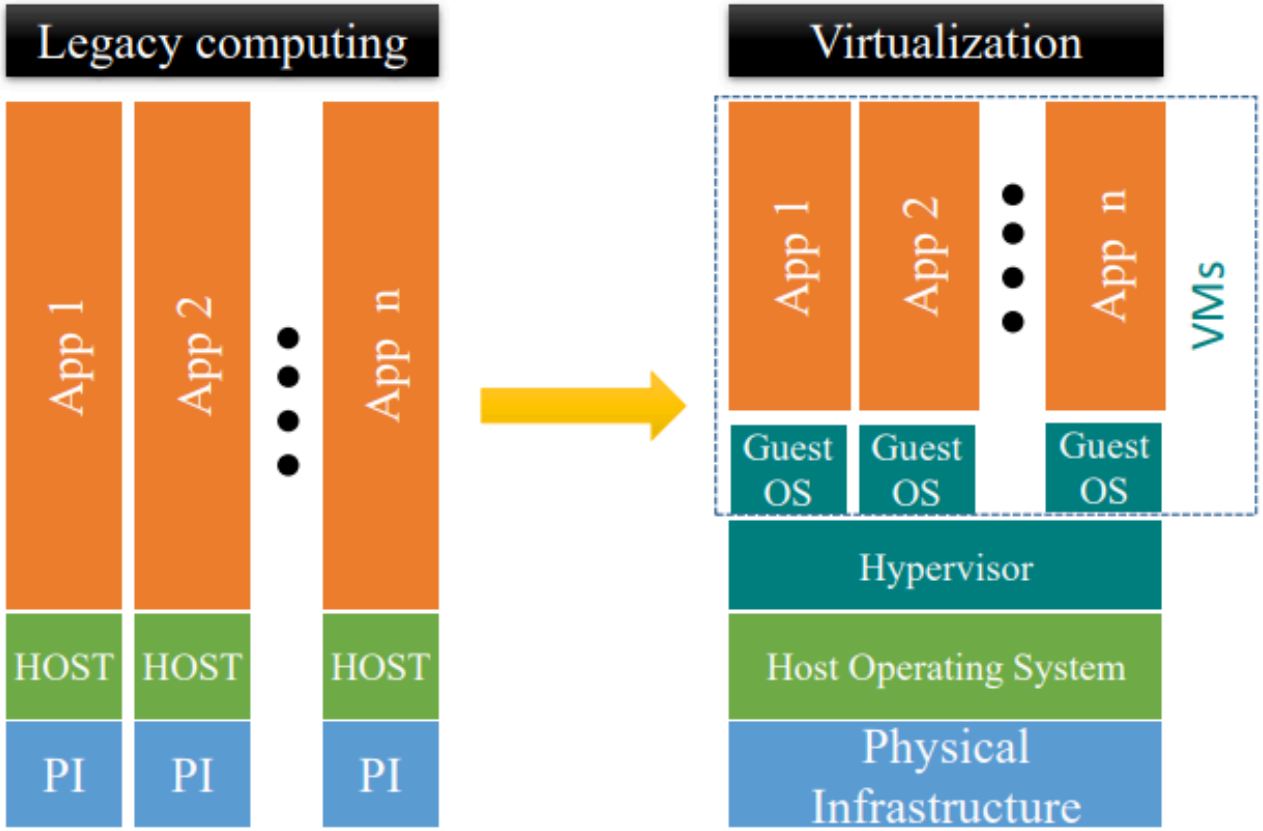
Nazih Errahel

erraheln@ex.istu.edu

OUTLINE

- What is Container?
  - Namespace & Cgroup
  - Docker
  - Basic commands
  - Data management

Journey of Virtualization



But VMs are suffering from

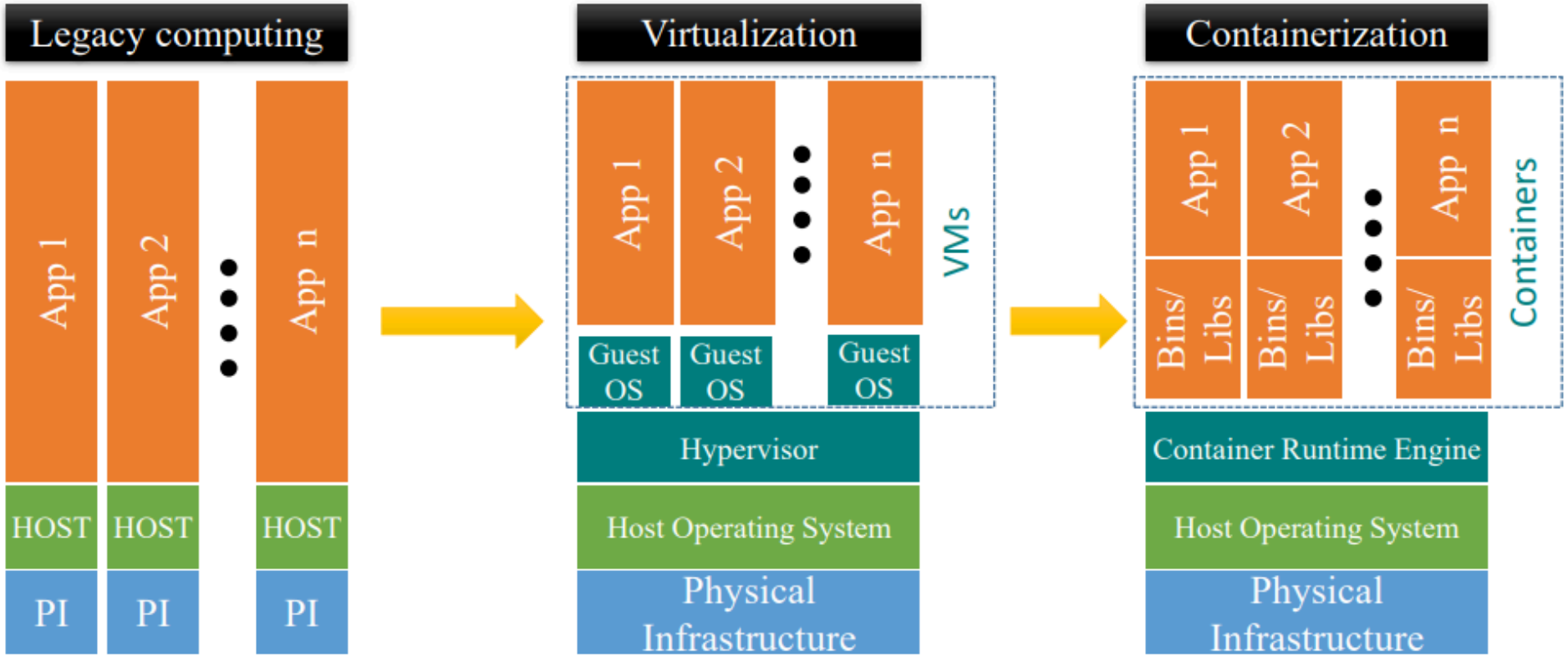
-Cold start

- Need more storage

  -Less number of virtual machines per PI

PI: Physical Infrastructure

Containerization…



- Lightweight
- Require less memory space

-Fast lunch time

- better resource utilization
- 1000s of containers can be loaded onto a Host

- Containerized apps share Host OS's kernel to execute work

Containers

-What is this?

Containers

What is a container?

- LXC, Linux container, is a Linux operating system-level virtualization method for running multiple isolated Linux based systems (user-space instances) on single host controlled and managed by ***Namespaces*** and ***Cgroups***.

- ***Namespaces***: Linux namespace partitions processes and system resources so that only processes in the same name group get access to name group resources and processes.
- ***Cgroups***: Originally contributed by Google, Cgroups(*Control Group*) is a Linux kernel concept that governs the isolation and usage of system resources, such as CPU & memory, for a group of processes.

Containers Benefits

Portability: not tied to or dependent upon the host operating system

Agility: capability to manage various kinds of changes during the development process. Able to respond quickly to the changes…

Speed: less start-up time, most lightweight…

Fault isolation: does not affect other containers…

Efficiency: less start-up time, smaller in capacity, allow more container to run

Security: due to isolation of applications

Src: https://www.ibm.com/cloud/learn/containerization

Containers Benefits -VMs vs Containers

Virtual Machines	Containers
Heavyweight	Lightweight
Limited performance	Native performance
Each VM runs in its own OS	All containers share the host OS
Hardware-level virtualization	OS virtualization
Startup time in minutes	Startup time in milliseconds
Allocates required memory	Requires less memory space
Fully isolated and hence more secure	Process-level isolation, possibly less secure

Docker containers are NOT VMs

- Fundamentally different architectures
- Fundamentally different benefits

Src: https://www.backblaze.com/blog/vm-vs-containers/

Containers – *Namespaces*

- Became part of Linux kernel since about 2002
- a feature of the Linux kernel that partitions kernel resources such that one set of processes sees one set of resources while another set of processes sees a different set of resources. -Wikipedia
- A Mechanism for resource isolation

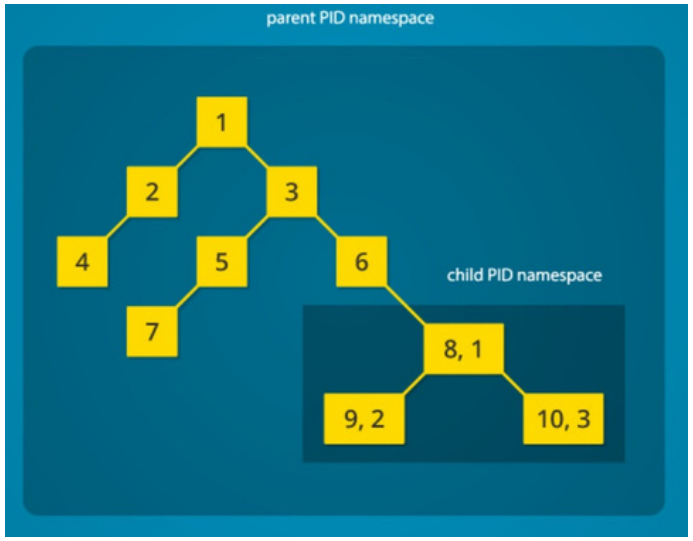
- More on Namespaces : https://man7.org/linux/man-pages/man7/namespaces.7.html

Src:
https://www.toptal.com/linux/separation-anxiety-isolating-your-system-with-linux-namespaces
https://blog.codecentric.de/en/2019/06/docker-demystified/

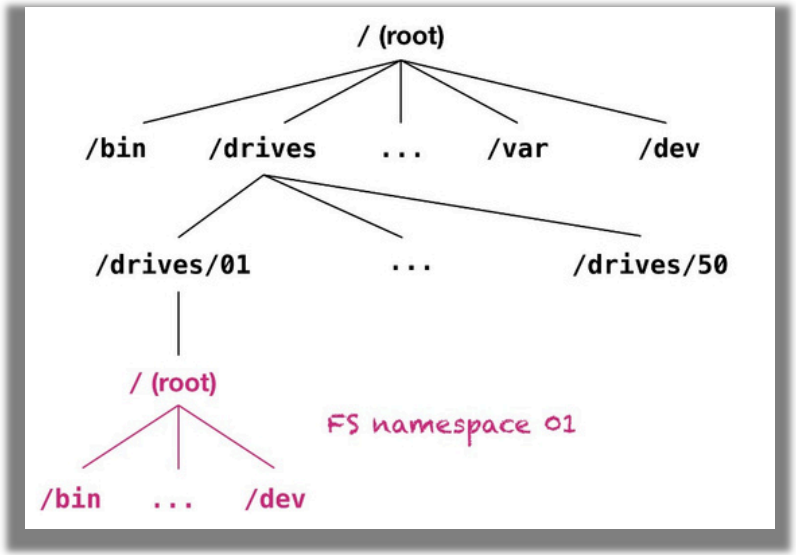
Containers –*Namespaces*

- Examples:PID (Process Id), MNT (Mountfile/folder), IPC, NET(Individual port and IP)

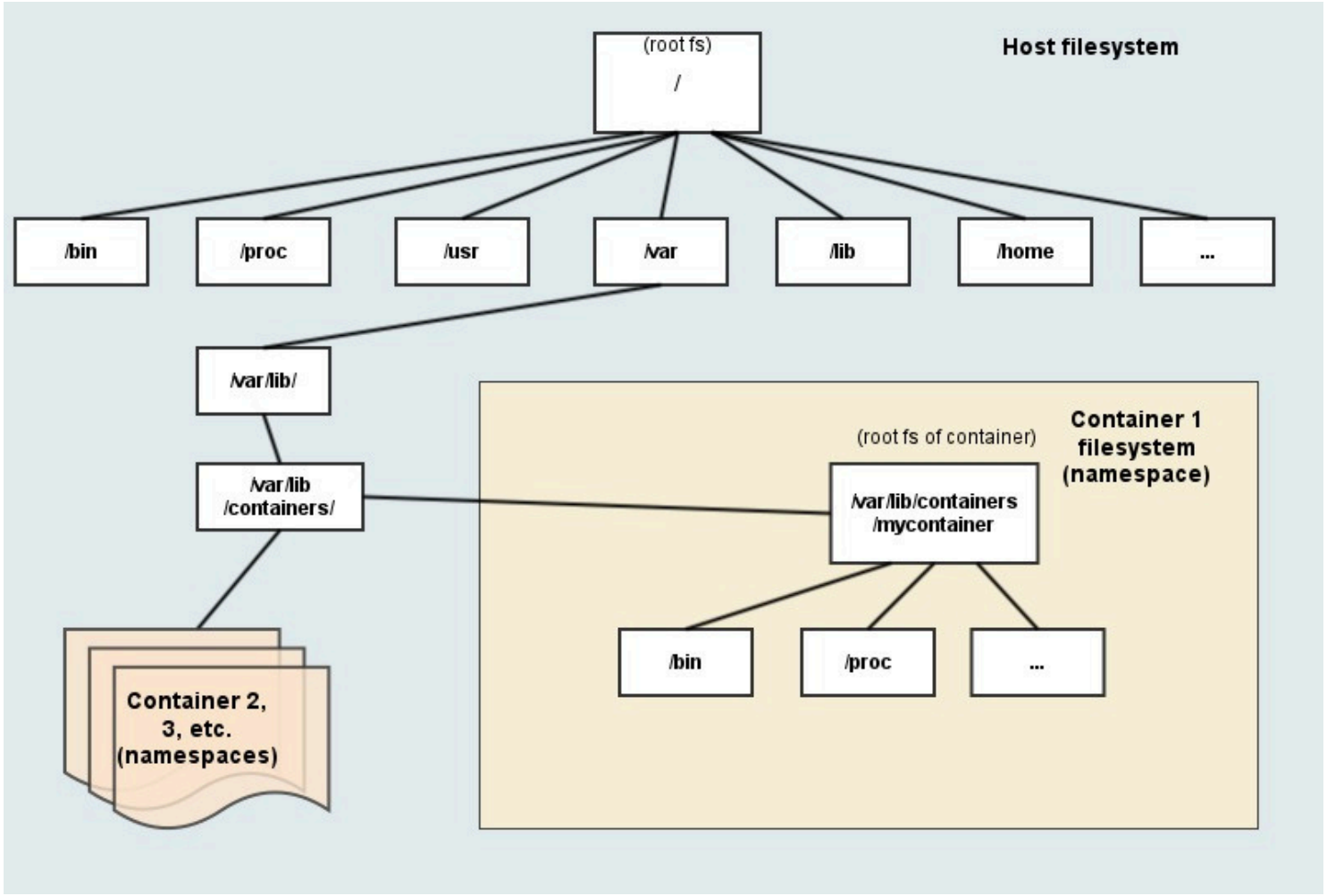
Process Id namespace



Filesystem namespace



Broad view of Filesystem namespace



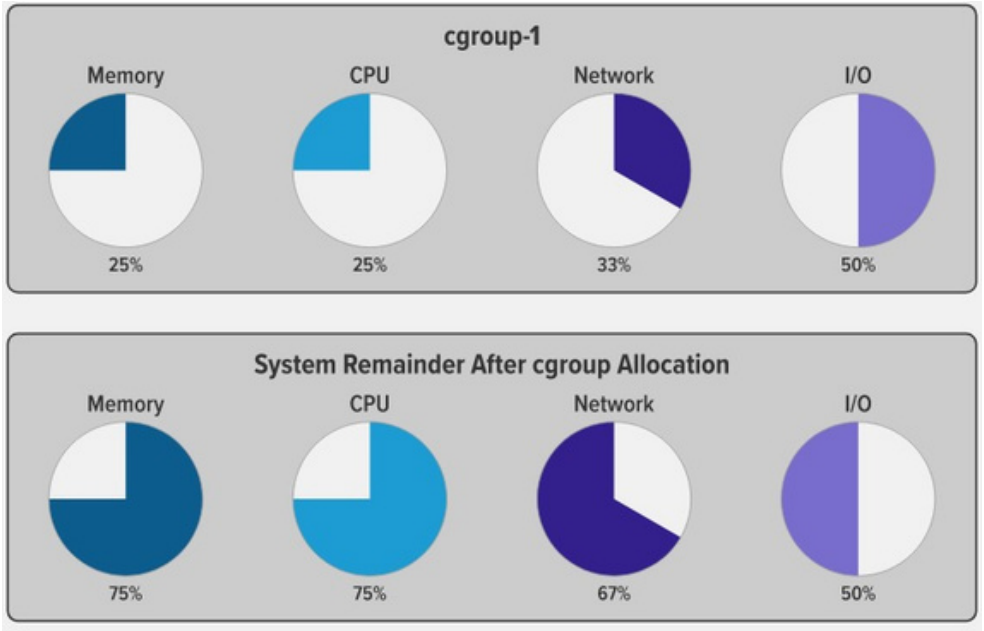
Containers –*Cgroups*

- ControlGroup
- "… cgroups, a feature for isolating and controlling resource usage (e.g., how much CPU and RAM and how many threads a given process can access) within the Linux kernel. …"
- Used to associate process with the system resource
  - Track which processes are using a particular system resource

https://www.nginx.com/blog/what-are-namespaces-cgroups-how-do-they-work/

Containers –*Cgroups*

Control Group : "… cgroups, a feature for isolating and controlling resource usage (e.g., how much CPU and RAM and how many threads a given process can access) within the Linux kernel. …"



Cgroups provide the following features:

Resource limits–limit how much of a particular resource memory or CPU, for example) a process can use.
Prioritization–control how much of a resource a process can use compared to processes in another cgroup…
Accounting–Resource limits are monitored and reported at the cgroup level.

https://www.nginx.com/blog/what-are-namespaces-cgroups-how-do-they-work/

Some container runtime platforms…

- Docker
- CoreOS rkt
- Mesos
- LXC
- OpenVZ
- Containerd
- Windows Server Containers
- Linux VServer
- Hyper-V Containers
- Unikernels
- Java containers

Some container runtime platforms…

- **Docker**
- CoreOS rkt
- Mesos
- LXC
- OpenVZ
- Containerd
- Windows Server Containers
- Linux VServer
- Hyper-V Containers
- Unikernels
- Java containers

Docker

Docker

- Docker container technology was launched in 2013 as an *open source* Docker Engine.

- Docker enterprise edition introduced in 2016 as a first commercial product.
- Docker is written in the **Go programming language**
- When you run a container, Docker creates a set of *namespaces* for that container.

Docker –terminologies/vocabulary

The Role of Images and Containers

Docker Image

Docker Container

Docker Image

- The basis of a Docker container
- Images are read only templates build from Dockerfile.

Docker Container:

- The image when it is running
- The standard unit for application service.
- Runs your application.

Docker –terminologies/vocabulary

Docker Engine

Creates, ships and runs Docker containers deployable on a physical or virtual, host locally, in a data center or cloud service provider

Registry Service (Docker Hub(Public) or Docker Trusted Registry(Private))

Cloud or server based storage and distribution service for your images

Basic Docker Commands

- docker --version
- docker pull <image name>
- docker run -it -d <image name>
- docker ps
- docker image ls
- docker exec -it <containerid> <command>
- docker stop <container id>
- docker kill <container id>
- docker login

Basic Docker Commands

Build

Build an image from the Dockerfile in the current directory and tag the image
```
docker build -t myimage:1.0 .
```

List all images that are locally stored with the Docker Engine
```
docker image ls
```

Delete an image from the local image store
```
docker image rm alpine:3.4
```

Share

Pull an image from a registry
```
docker pull myimage:1.0
```

Retag a local image with a new image name and tag
```
docker tag myimage:1.0 myrepo/myimage:2.0
```

Push an image to a registry
```
docker push myrepo/myimage:2.0
```

Run

Run a container from the Alpine version 3.9 image, name the running container "web" and expose port 5000 externally, mapped to port 80 inside the container.
```
docker container run --name web -p 5000:80 alpine:3.9
```

Stop a running container through SIGTERM
```
docker container stop web
```

Stop a running container through SIGKILL
```
docker container kill web
```

List the networks
```
docker network ls
```

List the running containers (add `--all` to include stopped containers)
```
docker container ls
```

Delete all running and stopped containers
```
docker container rm -f $(docker ps -aq)
```

Print the last 100 lines of a container's logs
```
docker container logs --tail 100 web
```

Src: https://www.docker.com/sites/default/files/d8/2019-09/docker-cheat-sheet.pdf

Dockerfile basic

- *Dockerfile*, a text file that includes specific keywords that dictate how to build a specific image.
- Docker builds images automatically by reading the instructions from a *Dockerfile.*
- An Example of *dockerfile*

```
FROM python:3.8-slim-buster
WORKDIR /app
COPY requirements.txt requirements.txt
RUN pip3 install -r requirements.txt
COPY . .
CMD [ "python3", "-m" , "flask", "run", "--host=0.0.0.0"]
```

FROM: creates a layer from the ubuntu:18.04 Docker image.
WORKDIR : sets the path where the command, defined with CMD, is to be executed.
COPY: adds files from your Docker client's current directory.
RUN: builds your application and make the environment ready.
CMD: specifies what command to run within the container.

Src: https://docs.docker.com/develop/develop-images/dockerfile_best-practices/

Dockerfile basic

- An Example of *dockerfile*

```
FROM python:3.8-slim-buster
WORKDIR /app
COPY requirements.txt requirements.txt
RUN pip3 install -r requirements.txt
COPY . .
CMD [ "python3", "-m" , "flask", "run", "--host=0.0.0.0"]
```

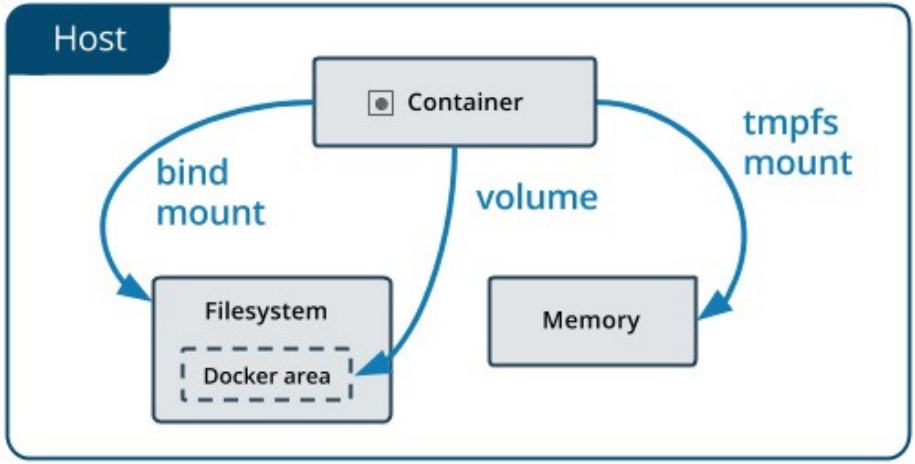
FROM: creates a layer from the ubuntu:18.04 Docker image.
WORKDIR : sets the path where the command, defined with CMD, is to be executed.
COPY: adds files from your Docker client's current directory.
RUN: builds your application with make.
CMD: specifies what command to run within the container.

Build the docker file: `docker build -t myflask .`

Src: https://docs.docker.com/develop/develop-images/dockerfile_best-practices/

Manage data in Docker

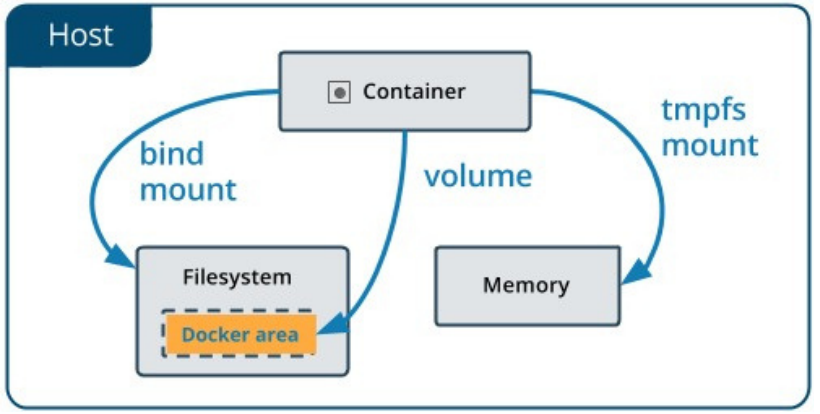
- By default all files created inside a container are stored on a writable container layer
  - The data doesn't persist when that container no longer exists



- Docker has two options for containers to store files in the host machine
  - *Volumes (/var/lib/docker/volumes/ on Linux)*
  - *bind mounts (anywhere on host)*

Manage data in Docker -Volumes

- *Volumes* are the preferred mechanism for persisting data generated by and used by Docker containers.
- Volumes are completely managed by Docker.



Create a volume:

```
docker volume create vol2
```

List volumes:

```
docker volume ls
```

Remove a volume:

```
docker volume rm vol2
```

Start a container with a volume

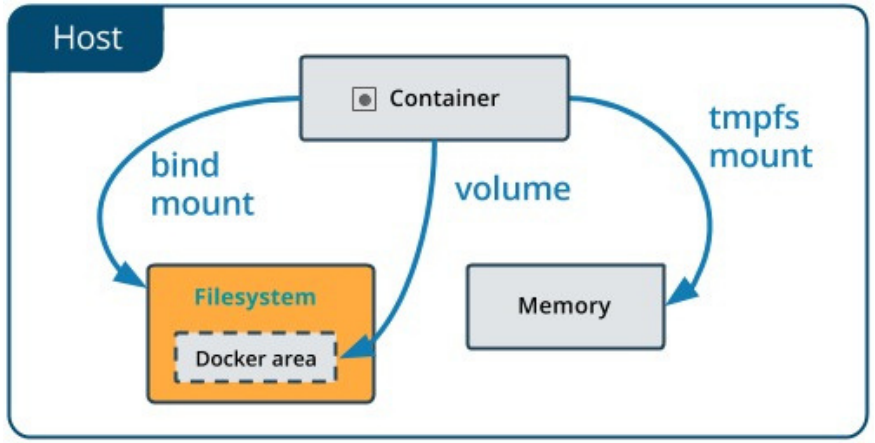
```
docker run -d --name mywebapp -v vol2:/app myFlask:latest
```

Manage data in Docker -bind mounts

- When you use a bind mount, a file or directory on the *host machine* is mounted into a container.
- The file or directory is referenced by its absolute path on the host machine.

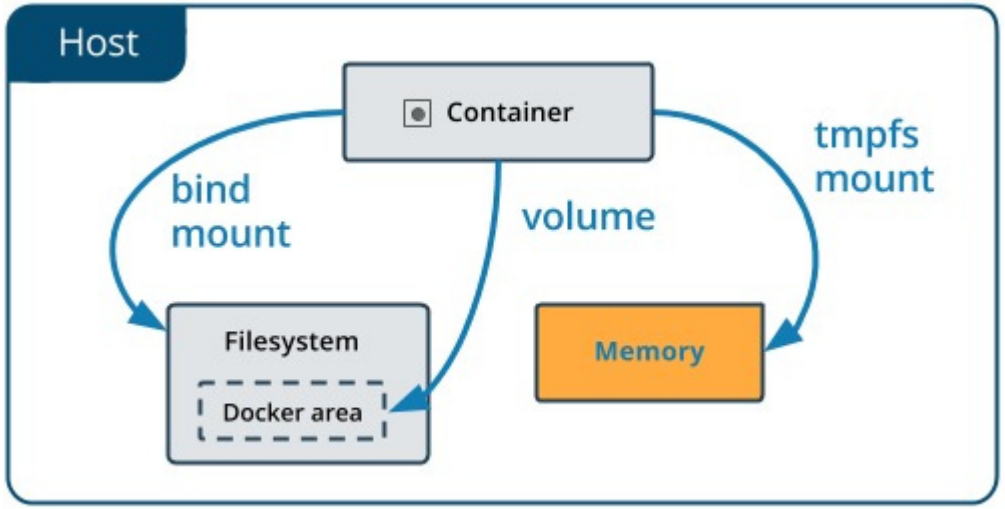
Start a container with a bind mount

```
docker run -d --name mywebapp -v "$(pwd)"/target:/app myFlask:latest
```



Manage data in Docker - **tmpfs mounts**

- *Volumes* and *bind mounts* let you share files between the host machine and container so that you can persist data even after the container is stopped.
- The third option *tmpfs mounts* option is only available to *Docker on Linux [src]*.
- As opposed to volumes and bind mounts, a tmpfs mount is *temporary*
- When the container stops, the tmpfs mount is removed, and files written there won't be persisted.
- *Useful* to temporarily store sensitive files



Start a container with a bind mount

```
docker run -d -it --name mywebapp --tmpfs /app myFlask:latest
```

Docker –in DevOps